



P3Q Corona +
Magnetar Kinds:
Cross-Family PQ Verify on
a Unified Precompile Slot

Zach Kelling

Lux Industries

2026-06-03

Abstract

LP-218 introduces P3Q at EVM slot 0x012205 as the PQ-verifier family for rollup-batch authentication, specifying the unified kind-byte dispatch ABI and the first verifier kind 0x01 (Pulsar / FIPS-204 ML-DSA-65 threshold). This paper is the per-kind specification of the other two production kinds: 0x02 Corona (Ring-LWE threshold, Ringtail eprint 2024/1113) and 0x03 Magnetar (FIPS-205 SLH-DSA threshold). Same slot, same ABI, three primitive kinds, three independent cryptographic families. Cross-family-maximum security (PQ-heavy mode per LP-217) requires that a single structural break in any one PQ family does not compromise the rollup; the three kinds reduce to Module-LWE, Ring-LWE, and hash collision resistance respectively. We document the 2026-06-03 architectural decompletion from sibling-slots to single-slot kind-byte dispatch, the per-kind wire layouts, the `P3QVerifier` Go interface that registers each kind, the per-kind gas calibration (Pulsar 50 000 gas, Corona 5 000 000 gas, Magnetar 30 000 gas), and the production parameter-set lock for Magnetar at SLH-DSA-192f after the 2026-06-04 Quasar 4-scheme matrix bench measured 192s at 795 ms/sign vs 192f at 36 ms/sign on Spark (a 22× reduction that restores the sub-1 s round budget). GPU dispatch for all three kinds is projected, not measured; on-chain verifier paths are wired only for Pulsar today, with Corona and Magnetar reserving their dispatch entries and returning `abiFalse` via `ErrKindNotImplemented` until their threshold libraries land.

Contents

1	Introduction	4
1.1	Why a single slot	4
1.2	The three kinds at a glance	4
1.3	Honest measurement state	5
1.4	Roadmap	5
2	Architectural Decision: Single Slot, Kind-Byte Dispatch	5
2.1	The prior sibling-slots model	5
2.2	The 2026-06-03 unification	6
2.3	Kind-byte dispatch	6
2.4	What this is not	7
2.5	Activation marker	7
3	Pulsar Kind (kind = 0x01)	7
3.1	Slot and ABI	7
3.2	Wire format	8
3.3	Verifier core	8
3.4	Gas cost	8
3.5	Domain separation	8
3.6	Performance	9
3.6.1	Note on GPU verify projection	9
3.7	Use case	9
4	Corona Kind (kind = 0x02)	9
4.1	Slot and ABI	9
4.2	Underlying scheme	9
4.3	Wire format	10
4.4	Verifier core	10
4.5	Gas cost	10
4.6	Performance	11
4.6.1	Scaling exponent	11
4.7	Use case	11

5	Magnetar Kind (kind = 0x03)	11
5.1	Slot and ABI	11
5.2	Underlying scheme	12
5.3	Wire format	12
5.4	Verifier core	12
5.5	Parameter set choice: 192f for production (locked 2026-06-04)	12
5.5.1	Production lock	13
5.6	Gas cost	13
5.7	Performance	13
5.8	Use case	13
6	Verifier Interface and Dispatch	14
6.1	The P3Q Verifier Go interface	14
6.2	Dispatch flow	14
6.3	Per-kind gas table	15
6.4	Validator registration	15
6.5	Failure modes	16
7	Composition With Sibling LPs	16
7.1	LP-217 cert profile modes and the P3Q kinds	16
7.1.1	Sample rollup-contract composition (Solidity)	16
7.2	LP-218 rollup-batch pattern	17
7.3	LP-300 schema registry	17
7.4	Cross-references	18
7.5	Backwards compatibility	18
7.6	Future work	18

1 Introduction

A post-quantum rollup architecture has two orthogonal concerns:

1. **The rollup-batch envelope pattern** – how a sequencer composes transactions, threshold-signs a state-root tuple, and posts a single `RollupBatchTx` that the parent L1 verifies inside normal block validation. This is the contribution of LP-218: the wire format, the parent-L1-verifies-the-sig pattern, the inherited cert mode, and the first P3Q kind (0x01 Pulsar).
2. **The per-kind verifier specifications** – the individual cryptographic primitives that can occupy a P3Q kind slot, their wire layouts, their gas calibrations, their security assumptions, and the operator semantics for composing them into a cross-family-maximum PQ posture. This is the contribution of LP-220.

LP-218 fixes the pattern and the slot. LP-220 fills in the remaining two kinds.

1.1 Why a single slot

A naive design would allocate one EVM precompile slot per PQ family: 0x012205 for Pulsar, 0x012206 for Corona, 0x012207 for Magnetar. The earlier draft of LP-220 did exactly that, treating the rollup-batch wrappers as sibling primitives. Three slots, three ABIs, three audit surfaces, three call-paths in EVM.

The 2026-06-03 decompilation collapses this into one slot with kind-byte dispatch. The motivations are not aesthetic:

- **Solidity callability is a UX surface.** A rollup contract that wants cross-family-maximum verify writes three `P3Q.verify(kind, ...)` calls at a single address. Three slots would mean three address constants, three ABI imports, three precompile registrations to track in the operator’s head. One slot keeps the developer surface coherent.
- **Audit surface compounds.** Three slots with three wrappers means three independent code paths around the threshold-signature verifier core – the wrappers carry their own structural parsing, length checks, gas billing, and domain-separation contexts. Each is an audit boundary. One slot dispatches into per-kind verifier cores via the `P3QVerifier` interface and leaves only the dispatch table to audit.
- **Future kinds add a byte, not a slot.** If a fourth PQ family enters the Lux stack (a hypothetical code-based signature with HQC-style structure, say), it lands as one new kind byte in the dispatch table and one new `P3QVerifier` implementation. No new precompile slot allocation, no new Solidity function shape, no new audit boundary at the EVM-side.
- **One and only one way per LP-200.** The ZAP Stack umbrella codifies the principle: every distinct concern has exactly one canonical entry point. “Verify a rollup-batch threshold signature on-chain” is one concern; the kind byte selects which primitive answers the question.

1.2 The three kinds at a glance

kind	Name	PQ family	Underlying scheme
0x01	Pulsar	Module-LWE	FIPS-204 ML-DSA-65 threshold
0x02	Corona	Ring-LWE	Ringtail two-round threshold
0x03	Magnetar	Hash-based	FIPS-205 SLH-DSA threshold

Table 1: The three production kinds at slot 0x012205.

A simultaneous structural break of all three would require independent advances in (i) Module-LWE cryptanalysis, (ii) Ring-LWE / ring-structured lattice cryptanalysis, and (iii) collision resistance of the SHA-2 / SHA-3 family. These three assumptions are widely understood to be mutually independent. The cross-family-maximum posture LP-217 PQ-heavy codifies

relies on that independence; LP-220 supplies the on-chain machinery to actually compose all three kinds inside a single rollup contract.

1.3 Honest measurement state

Throughout this paper, performance numbers are tagged **measured** or **projected**.

- **Measured** numbers come from the 2026-06-04 Quasar 4-scheme matrix bench (`lux/threshold/protocols`) on M1 Max (10-core darwin/arm64) and Spark (NVIDIA GB10 Grace Blackwell, 20-core linux/aarch64). Numbers are wall-clock per per-validator signing round at $N=\{16, 32, 64\}$, mean of 50 rounds.
- **Projected** numbers are CPU verify costs scaled by GPU speedup ratios from kernel-level micro-benchmarks. GPU dispatch is not exercised by any path in the matrix bench – `nvidia-smi dmon` mean `SM%` during the 2-hour Spark run was 11.9% (idle/background only). Two boundary gaps prevent dispatch: `lux-lattice.pc` is not installed on Spark (`find_package(MLX REQUIRED)` fails), and `lux_slhdsa_*` entry points are not yet exposed in `lux/accel/c_api.h`. Until both close, all GPU latency claims in this paper are projections.

The architectural decisions (slot, ABI, dispatch, parameter set choice) are not contingent on the GPU dispatch closing. They are contingent only on the CPU verifier correctness, which is byte- equal to the underlying NIST-standard (Pulsar/Magnetar) or to the Ringtail reference implementation (Corona) and is wired already.

1.4 Roadmap

Section 2 documents the 2026-06-03 decompaction in detail: the prior sibling-slots model, the unification onto 0x012205, and the relationship to LP-0120’s separate raw-primitive verifier slots. Sections 3–5 specify each kind in turn. Section 6 defines the `P3QVerifier` Go interface that registers each kind into the dispatch table. Section 7 ties the three kinds back into LP-217 cert profile modes and shows the rollup-contract composition pattern for the PQ-heavy posture.

2 Architectural Decision: Single Slot, Kind-Byte Dispatch

2.1 The prior sibling-slots model

The pre-decompaction draft of LP-220 (dated 2026-05-29) allocated three EVM precompile slots:

- 0x012205 – P3Q-Pulsar, wrapping the rollup-batch ABI around the Pulsar threshold-FIPS-204 verifier core.
- 0x012206 – P3Q-Corona, wrapping the rollup-batch ABI around the Corona (Ringtail) Ring-LWE threshold verifier core.
- 0x012207 – P3Q-Magnetar, wrapping the rollup- batch ABI around the Magnetar threshold-SLH-DSA verifier core.

Each slot reused the corresponding LP-0120 raw-primitive verifier as the underlying implementation (0x012204 raw Pulsar, 0x012206 raw Corona, 0x012207 raw Magnetar) and layered a rollup-batch-specific wrapper on top: structural parsing for the (`prev_root`, `new_root`, `batch_hash`) tuple, a separate FIPS context string per slot, separate gas calibration billed inside each wrapper.

The collision between LP-220’s rollup-batch wrappers and LP-0120’s raw-primitive verifiers at slots 0x012206 and 0x012207 was the first symptom that the design was overcomplicating two concerns:

1. “Verify a threshold signature of family F over a 32-byte digest” (the raw primitive concern, owned by LP-0120).
2. “Verify a rollup-batch envelope of family F where the digest is $H(\text{prev_root} \parallel \text{new_root} \parallel \text{batch_hash})$ and the FIPS context binds the rollup pattern” (the rollup-batch concern, owned by LP-218 / LP-220).

These are two distinct concerns. They deserve two distinct surfaces. But they do not deserve six precompile slots between them.

2.2 The 2026-06-03 unification

The decompilation allocates the rollup-batch concern exactly one slot – 0x012205 – and reuses the existing raw-primitive slots (0x012204, 0x012206, 0x012207) for the raw-primitive concern. The rollup-batch slot dispatches on a leading `kind` byte to per-primitive verifier implementations:

Slot	Concern	Wire	Owner
0x012202	Raw ML-DSA single-party	FIPS-204	LP-0120
0x012203	Raw SLH-DSA single-party	FIPS-205	LP-0120
0x012204	Raw Pulsar (Module-LWE threshold)	FIPS-204 ctx + sig	LP-0120
0x012205	P3Q family (rollup-batch ABI)	kind-byte dispatch	LP-218 + LP-220
0x012206	Raw Corona (Ring-LWE threshold)	Ringtail ctx + sig	LP-0120
0x012207	Raw Magnetar (SLH-DSA threshold)	FIPS-205 ctx + sig	LP-0120

Table 2: Precompile slot ownership after the 2026-06-03 decompilation. Slots 0x012204, 0x012206, 0x012207 remain owned by LP-0120 for the raw-primitive non-rollup-batch use case; LP-218 + LP-220 own 0x012205 as the P3Q rollup-batch family.

A Solidity contract that needs Corona-kind verify for a rollup-batch envelope calls `P3Q.verify(0x02, sig, h, pk)` at 0x012205. A Solidity contract that needs raw Corona primitive verify – a non-rollup-batch use case, e.g. a custody attestation contract verifying an off-chain Corona-signed certificate – calls the LP-0120 raw-primitive verifier at 0x012206 directly. Two distinct callers, two distinct surfaces, two distinct slots, zero overlap.

The P3Q family ABI is the single canonical entry point for rollup-batch verification across all PQ kinds. This honors LP-200 § “One and only one way to do everything” [4].

2.3 Kind-byte dispatch

The unified 0x012205 precompile reads a 1-byte `kind` discriminator at offset 0 of calldata and routes to one of three (today) registered verifier implementations. The dispatch is at runtime, populated by package `init()` functions; the precompile entry-point is one function in one file.

The full dispatch logic from the reference implementation at `lux/precompile/p3q/contract.go::Run`:

```
kind := input[0]
switch kind {
case KindPulsar:      // 0x01
    // fall through to FIPS 204 ML-DSA verify path
case KindCorona,      // 0x02
    KindMagnetar:      // 0x03
    // Reserved kinds -- verifier not yet wired. Return abiFalse
    // (the LP-218 "no revert on cryptographic failure" contract)
    // so callers still see a consistent surface; once the
    // lattice/hash threshold libraries land, this branch routes
    // to its own verifier.
    return abiFalse(), remaining, nil
default:
    // Unknown kind -- abiFalse, same as any malformed input.
    return abiFalse(), remaining, nil
}
```

Three properties hold by construction:

- **Constant-time over the kind byte.** The switch is on a PUBLIC field (the calldata kind byte is observable on-chain). Branching on it does not constitute a timing side channel against any secret.
- **No revert on cryptographic failure.** An unknown kind, an unwired reserved kind, a malformed sig, a FIPS-204 reject – all return `abiFalse` (a 32-byte zero-encoded EVM bool) with no Go error. The Solidity caller observes `false` via the return word and decides whether to revert. This matches LP-218 § “Failure modes.”
- **Uniform gas billing.** The `RequiredGas` function is monotonic in input length and does NOT depend on the kind. A short malformed input pays the base gas, a well-formed N-byte input pays `base + N * PerByteGas`. No reduced price for early rejection – uniform billing prevents a gas-side channel from leaking which validation stage rejected the input.

Per-kind gas *rates* differ (Pulsar 50 000 gas, Corona 5 000 000 gas, Magnetar 30 000 gas); the resolution happens inside the per-kind `P3QVerifier.GasCost()` method described in section 6. The `0x012205` precompile multiplexes those rates over a single observable address.

2.4 What this is not

It is worth being explicit about two design decisions LP-220 does *not* make:

1. **It is not a generic PQ-signature precompile.** The three kinds are not interchangeable. A Pulsar signature submitted under `kind=0x02` (Corona) will fail structurally – Pulsar sigs are 3309 B, Corona sigs are ~ 33 KB, and the verifier core for kind `0x02` is the Ringtail Ring-LWE verifier (which does not parse FIPS-204 output). The kind byte selects a specific verifier; it does not abstract over them.
2. **It is not a backwards-compatible upgrade of the sibling-slots draft.** There is no LP-220-v1 deployment in the wild; the activation marker `2025-12-25T16:20:00-08:00` places the unified-slot ABI at network genesis. Existing test corpora that target `0x012206` / `0x012207` for rollup-batch wrappers are obsolete (those slots are now LP-0120 raw-primitive surfaces with different wire format); they must be re-targeted to `0x012205` with the appropriate kind byte. No compatibility shim exists.

2.5 Activation marker

`activates: 2025-12-25T16:20:00-08:00`
`activates-unix: 1766708400`

The P3Q family ABI at slot `0x012205` with kinds `0x01`, `0x02`, `0x03` registered in the dispatch table is callable from the genesis block of the new final Lux network. Rollup deployments that target PQ-strict or PQ-heavy mode call `P3Q.verify(kind, ...)` from height 0 with the appropriate kind byte (subject to per-kind verifier wiring, see sections 4–5).

3 Pulsar Kind (kind = 0x01)

3.1 Slot and ABI

`0x012205` with leading `kind=0x01`. The Solidity shim is:

```
// EVM precompile at slot 0x012205, kind = 0x01
bool ok = P3Q.verify(
    0x01, // Pulsar kind
    pulsarSig, // FIPS-204 ML-DSA-{44|65|87} sig
    messageHash, // H(prev_root || new_root || batch_hash)
    groupPubKey // Pulsar group public key
);
```

The kind byte at offset 0 selects dispatch into the Pulsar verifier; the mode byte at offset 1 selects which FIPS-204 parameter set is in play.

3.2 Wire format

The full 5303 B canonical input for ML-DSA-65 (the production target):

Offset	Size	Field
0	1	kind = 0x01
1	1	mode (0x44/0x65/0x87 = ML-DSA-44/65/87)
2	4	sigLen (BE32)
6	sigLen	pulsarSig (3309 B for ML-DSA-65)
6+sigLen	4	pkLen (BE32)
10+sigLen	pkLen	groupPubKey (1952 B for ML-DSA-65)
10+sigLen+pkLen	32	messageHash

Table 3: Pulsar-kind wire format. Total at ML-DSA-65: $1+1+4+3309+4+1952+32 = 5303$ B.

Trailing bytes after the messageHash are accepted as padding to match the convention used by the sibling STARK-FRI precompile; padding does not affect verification.

3.3 Verifier core

The Pulsar verifier core IS the FIPS-204 ML-DSA verifier. Pulsar signatures serialize byte-identically to FIPS-204 ML-DSA signatures per LP-073 [2] – the Pulsar threshold ceremony’s Class-N1 claim is that the aggregated signature is byte-equal to a single-party FIPS-204 ML-DSA signature on the same message and group public key. Any conformant FIPS-204 verifier suffices; the reference implementation at `lux/precompile/p3q/contract.go::Run` dispatches via `mldssa.PublicKeyFromBytes(...)` + `pk.VerifySignatureCtx(...)`, which is a thin alias over the cloudflare/circl FIPS-204 implementation.

The Pulsar kind is the only kind with a fully-wired verifier today. Corona (0x02) and Magnetar (0x03) reserve their dispatch entries and return `abiFalse` via the `ErrKindNotImplemented` path until their threshold libraries land. See sections 4 and 5.

3.4 Gas cost

50 000 gas base + 10 gas/byte per-byte adder. At the canonical 5303 B input, total gas is $50000 + 5303 \times 10 = 103\,030$ gas. This is well under the 12 M C-Chain block-gas-limit (~116 verifies per block; per the audit at `contract.FeeConfigReporter`). The gas calibration is $10\times$ the BLS pairing verify charge of 5000 gas, reflecting lattice verification’s inherent cost (NTT inverse + matrix-vector multiplication mod q).

3.5 Domain separation

Every Pulsar-kind verify binds the FIPS-204 context string `lux-evm-precompile-p3q-v1`. Mismatched context between signer and verifier causes the FIPS-204 verifier to reject. This prevents cross-precompile replay: a Pulsar signature minted for the LP-0120 raw-Pulsar precompile at 0x012204 (which uses context `lux-evm-precompile-pulsar-v1`) cannot be re-submitted to the rollup-batch P3Q path at 0x012205.

Metric	M1 Max	Spark	Source
Per-validator Pulsar sign at $N=64$	391 ms	585 ms	matrix bench (measured) [6]
CPU verify (single sig)	$\sim 80 \mu\text{s}$	$\sim 80 \mu\text{s}$	circ benchmark (measured)
Blackwell sm_120 verify	—	$\sim 50 \mu\text{s}$	projected (LP-203 kernel)
Blackwell sm_120 batched ($N=64$)	—	$\sim 5 \text{ ms}$ aggregate	projected

Table 4: Pulsar kind performance. Sign times are measured per the Quasar 4-scheme matrix bench (mean of 50 rounds at $N=64$); verify times tagged “projected” have not been exercised in the matrix bench (GPU dispatch boundary gap, see section 3.6.1).

3.6 Performance

3.6.1 Note on GPU verify projection

The Blackwell verify numbers cited in earlier LP-203 drafts ($\sim 50 \mu\text{s}$ single, $\sim 5 \text{ ms}$ batched) are kernel-level micro-benchmarks, not end-to-end measurements through the precompile entry-point. The matrix bench captured 4595 `nvidia-smi dmon` samples during the 2-hour Spark run: mean `SM%` = 11.9% (idle/background only). Until `lux-lattice.pc` is installed on Spark and `lux_pulsar_verify_batch` is exposed in `lux/accel/c_api.h`, GPU verify timings remain projections.

3.7 Use case

Pulsar-kind is the default PQ-fast verifier for any rollup that wants Module-LWE-grade PQ assurance on its batch envelope without paying the cross-family premium of Corona or Magnetar. Per LP-217, PQ-fast (BLS + Pulsar) is the recommended mode for payment rails, gaming, and DEX matching. The gas cost is $\sim 55\,000$ per batch (BLS + Pulsar), well below the $\sim 5 \text{ M}$ cost of adding Corona.

LP-218 specifies the Pulsar kind in detail as part of the rollup-batch envelope pattern; this section restates the Pulsar-kind facts in the LP-220 per-kind structure so the three kinds are commensurable in one document.

4 Corona Kind (kind = 0x02)

4.1 Slot and ABI

0x012205 with leading `kind=0x02`. Same physical precompile address as Pulsar (kind 0x01) and Magnetar (kind 0x03); dispatch resolves to the Corona `P3QVerifier` implementation.

```
// EVM precompile at slot 0x012205, kind = 0x02
bool ok = P3Q.verify(
    0x02,                // Corona kind
    coronaSig,           // serialized Corona threshold sig
    messageHash,         // H(prev_root || new_root || batch_hash)
    groupPubKey          // Corona group public key
);
```

4.2 Underlying scheme

Corona is the Lux deployment of the Ringtail two-round lattice threshold signature scheme [1]. Hardness reduces to Ring-LWE in dimension $n = 1024$, modulus $q \approx 2^{40}$, and the standard Ringtail two-round distributed signing protocol.

Corona’s signature is *not* byte-equal to any NIST standard. ML-DSA (FIPS-204) is the Module-LWE relative; Corona is the Ring-LWE relative; they share the broader lattice family but sit on different rings with different polynomial commitment structures. A Ring-LWE break

does not imply a Module-LWE break and vice-versa; this is the cross-family-independence property the cross-family-maximum posture relies on.

4.3 Wire format

Offset	Size	Field
0	1	kind = 0x02
1	1	mode = 0x01 (reserved Corona parameter set)
2	4	sigLen (BE32)
6	sigLen	coronaSig (~33 052 B)
6+sigLen	4	pkLen (BE32)
10+sigLen	pkLen	groupPubKey
10+sigLen+pkLen	32	messageHash

Table 5: Corona-kind wire format. Sig size is the dominant cost (~ 33 KB); the rest mirrors the Pulsar layout to keep parsing uniform across kinds.

The mode byte is reserved at 0x01; future Corona parameter set additions (a higher-security Ring-LWE variant) get new mode bytes without changing the wire-format skeleton.

4.4 Verifier core

The Go reference implementation is planned at `lux/standard/precompiles/p3q_family/corona.go` and will wrap `luxfi/corona/sign.Verify`. The core is bespoke – not FIPS-204 byte-equal – because Corona is a separate Ring-LWE construction, not a NIST standard.

Status as of 2026-06-04: the verifier core is not yet wired into the P3Q dispatch table. The reference `p3q.Run` routes `kind=0x02` to `abiFalse` via the `ErrKindNotImplemented` path:

```
case KindCorona, KindMagnetar:
    // Reserved kinds -- verifier not yet wired. Return abiFalse
    // ...
    return abiFalse(), remaining, nil
```

Solidity callers MAY emit `P3Q.verify(0x02, ...)` from network genesis; the call charges the per-kind gas (see below) and returns `false` until the verifier lands. This is the LP-218 “no revert on cryptographic failure” contract: `false` on the return word is the universal signal for “not verified for any reason,” and reserved-but-not-wired joins “malformed input” and “FIPS reject” under that umbrella.

4.5 Gas cost

5 000 000 gas – $\sim 100\times$ the Pulsar kind’s 50 000. Returned by the Corona `P3QVerifier.GasCost()` entry. Why the disparity?

1. **CPU verify cost is heavier.** The Corona verifier requires NTT inverse over the Ring-LWE ring + polynomial multiplication mod q with the larger Ring-LWE coefficients (~ 33 KB sig vs Pulsar’s 3.3 KB).
2. **Calibration uses the CPU verify cost as the baseline.** LP-218’s calibration discipline assumes a CPU-class verifier – validators are not required to have Blackwell GPUs. Under that assumption, Corona’s effective per-validator verify cost is the parity audit’s measured ~ 1.6 ms CPU, which is approximately $141\times$ the Pulsar CPU verify time (~ 80 μ s). The gas charge rounds to $100\times$ Pulsar’s for clean accounting: 5 M gas.
3. **Block-budget headroom.** At 12 M block gas and 5 M per Corona verify, the limit is ~ 2 Corona verifies per block – this is the bottleneck for very high- density PQ-strict rollup throughput. The cost cliff is intentional; the LP-217 mode ladder uses it to price cross-family insurance correctly.

Operators with all-Blackwell validator sets may petition for a **Blackwell-discounted Corona gas charge** in a future amendment; LP-220 does not codify it. The conservative CPU-baseline gas charge applies until validator-hardware uniformity is established. This is the same calibration discipline LP-218 uses for the Pulsar kind.

4.6 Performance

Metric	M1 Max	Spark	Source
Per-validator Corona sign at $N=64$	~30 ms	~30 ms	matrix bench (measured) [6]
CPU verify (single sig)	~1.6 ms	~1.6 ms	parity audit (measured)
Blackwell sm_120 verify	—	~15 ms	projected (LP-203 kernel) [5]
Blackwell sm_120 batched ($N=64$)	—	~50 ms aggregate	projected

Table 6: Corona kind performance. Sign and CPU-verify times are measured per the parity audit; Blackwell verify timings remain projections until the GPU dispatch boundary closes (see section 3.6.1).

4.6.1 Scaling exponent

The parity audit measured Corona-leg signing cost growing super-linearly in N . The measured exponent across $N \in \{16, 32, 64\}$ is approximately $T(N) \approx c \cdot N^{1.5-1.7}$, depending on the host platform. This is not an $O(N)$ scheme and should not be described as such in downstream documents; the Ring-LWE distributed signing protocol’s full-rank check and Gaussian-hash PRF carry costs that grow faster than linearly in the participant count. The measured exponent is the operative number for sequencer capacity planning, not the naive-amortization figure.

4.7 Use case

A rollup that wants **Ring-LWE family independence** from Module-LWE configures its sequencer to sign each batch with both a Pulsar threshold sig AND a Corona threshold sig. The rollup contract on Z-Chain calls `P3Q.verify(0x01, ...)` and `P3Q.verify(0x02, ...)` at the same precompile address `0x012205` in sequence inside its `validateBatch` function; both must return `true` for the batch to be accepted.

A break of Module-LWE that compromises Pulsar does not compromise Corona; the rollup remains cryptographically anchored. The cost is ~5 050 000 gas per batch (Pulsar 50 000 + Corona 5 000 000). Acceptable for value-bearing rollups (custody, treasury, regulated securities); prohibitive for high-volume rollups (gaming, social).

This is the LP-217 PQ-strict posture made operationally callable.

5 Magnetar Kind (kind = 0x03)

5.1 Slot and ABI

`0x012205` with leading `kind=0x03`. Same physical precompile address as Pulsar (kind `0x01`) and Corona (kind `0x02`); dispatch resolves to the Magnetar `P3QVerifier` implementation.

```
// EVM precompile at slot 0x012205, kind = 0x03
bool ok = P3Q.verify(
    0x03,                                // Magnetar kind
    magnetarSig,                         // FIPS-205 SLH-DSA threshold sig
    messageHash,                         // H(prev_root || new_root || batch_hash)
    groupPubKey                          // Magnetar group public key
);
```

5.2 Underlying scheme

Magnetar is the Lux deployment of FIPS-205 SLH-DSA [7] under a Lux-defined threshold-aggregation wrapper specified in LP-181 [3]. The wrapper produces a signature byte-identical to a single-party FIPS-205 SLH-DSA signature on the same message and group public key; the verifier core is therefore the FIPS-205 SLH-DSA verifier with no threshold-specific logic.

SLH-DSA is hash-based: hardness reduces to the collision resistance of the underlying hash function (SHA-256 or SHA-3 configurations). This is the only PQ family in P3Q whose hardness does not depend on lattice assumptions; Magnetar is the cross-family last-line PQ kind.

5.3 Wire format

Offset	Size	Field
0	1	kind = 0x03
1	1	mode = 0x01 (reserved Magnetar parameter set)
2	4	bitmapLen (BE32)
6	bitmapLen	participant bitmap (set bit = participating)
6+blen	4	sigLen (BE32)
10+blen	sigLen	slhSig (~35 664 B for 192f)
10+blen+slen	4	pkLen (BE32)
14+blen+slen	pkLen	groupPubKey
14+blen+slen+pkLen	32	messageHash

Table 7: Magnetar-kind wire format. The participant bitmap is specific to the Magnetar kind – the threshold-aggregation wrapper needs to know which signers contributed in order to re-derive the aggregated public key for verification. Pulsar (0x01) and Corona (0x02) carry that information inside the sig structure; Magnetar’s hash-tree structure puts it in the wire envelope.

5.4 Verifier core

The Go reference implementation is planned at `lux/standard/precompiles/p3q_family/magnetar.go` and will wrap `luxfi/magnetar/verify`. The core is FIPS-205 SLH-DSA conformant (any conformant FIPS-205 verifier suffices).

Status as of 2026-06-04: same as Corona, the verifier core is not yet wired into the P3Q dispatch table; `p3q.Run` routes `kind=0x03` to `abiFalse` via the `ErrKindNotImplemented` path. Solidity callers MAY emit `P3Q.verify(0x03, ...)` from network genesis; the call charges the per-kind gas and returns `false` until the verifier lands.

5.5 Parameter set choice: 192f for production (locked 2026-06-04)

SLH-DSA at the 192-bit security level has two parameter sets; the production deployment is locked to the “fast” variant after the 2026-06-04 measurement.

Parameter	Sig size	Sign (Spark)	Sign (M1 Max)	Production fit
SLH-DSA-192s (small)	~16 KB	795 ms	1575 ms	breaks 1 s round budget
SLH-DSA-192f (fast)	~35 KB	36 ms	71 ms	fits LP-205 round budget

Table 8: SLH-DSA 192s vs 192f per-validator sign times, measured in the Quasar 4-scheme matrix bench (`/tmp/quasar-matrix-bench`, 2026-06-04, mean of 50 rounds at $N=64$). 192f is $22\times$ faster than 192s on Spark. Source [6].

The matrix bench measured Magnetar-192s as the wall-clock bottleneck of every PQ-heavy cell. At $N=64$ on Spark, PQ-heavy total wall-clock with 192s was 1.33 s – blowing past the 1 s

sub-second-finality target. The bottleneck is the per-validator SLH-DSA sign, not the threshold aggregation, not the verifier, not the network – it is the inherent FORS + WOTS+ hash-tree construction cost on the larger 192s tree.

Switching to 192f drops the per-validator sign from 795 ms to 36 ms ($22\times$ faster) at the cost of a $\sim 2.2\times$ larger signature (~ 35 KB vs ~ 16 KB). For a PQ-heavy rollup posting one Magnetar verify per batch, the extra 19 KB on the wire is negligible against the 0.76 s latency saved.

5.5.1 Production lock

LP-220-conformant deployments **MUST** configure Magnetar with SLH-DSA-192f. The threshold-init API at `lux/threshold/protocols/magnetar/` accepts a parameter- set argument; PQ-heavy chains pass `magnetar.ParamSetFast192`. Operators choosing 192s for the $2.2\times$ smaller signature opt into multi-second finality and MUST run `mode=PQ-strict` (no Magnetar leg) for the sub-second tier; 192s is reserved for long-archival anchoring use cases that don’t require fast finality.

Verification cost is symmetric between the two sets (~ 1.92 ms CPU); the parameter set affects only the wall-clock of the per-validator signing ceremony, not the on-chain verify cost. The Magnetar kind’s gas charge is 30 000 gas regardless of parameter set (signature size on the wire varies but the verifier work is bounded by the same hash-tree-walk constant).

5.6 Gas cost

30 000 gas – comparable to the Pulsar kind’s 50 000. SLH-DSA verify is fast (~ 1.92 ms CPU per the parity audit – comparable to Corona’s ~ 1.6 ms CPU but with very different gas cost) because the verifier walks a hash tree rather than performing lattice NTT inverse.

The gas calibration reflects the modest CPU verify cost relative to Corona; Magnetar verify is dominated by memory bandwidth on the WOTS+ tree-walk, which is a tight loop on SHA-256 / SHA-3 hash function evaluation that modern CPUs execute efficiently.

5.7 Performance

Metric	M1 Max	Spark	Source
Per-validator sign 192f at $N=64$	71 ms	36 ms	matrix bench (measured) [6]
Per-validator sign 192s at $N=64$	1575 ms	795 ms	matrix bench (measured)
CPU verify (single sig)	~ 1.92 ms	~ 1.92 ms	parity audit (measured)
Blackwell sm_120 verify	—	~ 50 ms	projected (LP-203 kernel) [5]
Blackwell sm_120 batched ($N=64$)	—	~ 80 ms aggregate	projected

Table 9: Magnetar kind performance. The 192s/192f signing delta is the dominant operational consideration; verify cost is symmetric between sets. Blackwell verify timings are projections.

5.8 Use case

A rollup that wants **hash-based family independence** from lattice families configures its sequencer to sign each batch with a Magnetar threshold sig in addition to Pulsar (and possibly Corona). The rollup contract calls `P3Q.verify(0x01, ...)` then `P3Q.verify(0x03, ...)` at slot `0x012205`.

SLH-DSA security relies only on the collision resistance of its underlying hash function; it is immune to advances in lattice cryptanalysis and would survive even a complete break of Module-LWE and Ring-LWE. Magnetar is the **last-line PQ** – the kind of choice for sovereign-archival commitments, long-tenor custody attestations, cross-chain bridges, and regulator-facing

audit trails. The cost (30 000 gas per verify + ~ 35 KB signature on the wire) is reasonable for low-frequency, high-value batches.

6 Verifier Interface and Dispatch

6.1 The P3QVerifier Go interface

The unification of the three kinds onto one slot is mediated by a Go-side interface at `lux/standard/precompile`. The interface is the single audit boundary for adding a kind: implement the four methods, register at `init()`, done.

```
// P3QVerifier is the per-kind verifier abstraction.
//
// Each P3QVerifier implementation is registered at process boot
// into a kind-byte keyed dispatch table loaded by the single
// P3Q precompile at slot 0x012205.
type P3QVerifier interface {
    // Verify returns true iff sig is a valid threshold signature
    // from groupPubKey on messageHash. The implementation must
    // be constant-time over secret inputs (there are none; both
    // sig and pk are PUBLIC calldata) and must not panic on
    // malformed input -- malformed inputs return false.
    Verify(sig []byte, messageHash [32]byte, groupPubKey []byte) bool

    // GasCost returns the per-call precompile gas charge for
    // this kind. The P3Q dispatcher charges this amount BEFORE
    // calling Verify, so an out-of-gas caller never observes the
    // verifier's behavior on its specific input.
    GasCost() uint64

    // Kind returns the calldata discriminator byte:
    // 0x01 = Pulsar      (Module-LWE)
    // 0x02 = Corona      (Ring-LWE)
    // 0x03 = Magnetar    (hash-based)
    Kind() byte

    // PrimitiveName returns "pulsar", "corona", or "magnetar".
    // Used for metric labels, log lines, and audit trails.
    PrimitiveName() string
}
```

The interface is minimal by design: four methods, three of which are constant-returning getters. The verifier core lives in the `Verify` method; the gas calibration lives in `GasCost`; the dispatch discriminator lives in `Kind`. There is no shared base class, no inheritance, no mutable verifier state – each verifier is value-typed and independently complete (composition over inheritance per LP-200).

6.2 Dispatch flow

When the `0x012205` precompile is called:

1. Read the leading `kind` byte from calldata (offset 0).
2. Look up the registered `P3QVerifier` for that kind in the kind-keyed dispatch table.
3. Charge `Verifier.GasCost()` (per-kind, returned by the impl). The uniform per-byte adder is added separately by the precompile's `RequiredGas`.
4. Parse the wire format down to (`sig`, `messageHash`, `groupPubKey`) using the layout appropriate to the kind (the per-kind size constants are baked into the implementation; the framing – BE32 length fields, trailing 32-byte `messageHash` – is uniform).
5. Call `Verifier.Verify(sig, messageHash, pk)`.
6. Return the boolean result encoded as a 32-byte EVM-ABI `bool` (`abiTrue` or `abiFalse`).

This is the single point of truth for the family. Three verifier implementations, one slot, one dispatch path, one ABI shape, one audit surface. Adding a fourth PQ-family kind (a

hypothetical code-based threshold via HQC-Sig variants, for example) is one new `P3QVerifier` impl and one new `kind` byte registered in the same dispatch table; no new precompile slot, no new Solidity function shape, no new audit boundary at the EVM-side.

6.3 Per-kind gas table

kind	Name	Gas charge	Verify CPU
0x01	Pulsar	50 000	~80 μ s
0x02	Corona	5 000 000	~1.6 ms
0x03	Magnetar	30 000	~1.92 ms

Table 10: Per-kind gas table. Corona’s 5 M charge is the $\sim 100\times$ multiplier over Pulsar reflecting the CPU verify-cost ratio; Magnetar’s 30 000 reflects the fact that the SLH-DSA hash-tree walk is memory-bandwidth-bound and parallelizes harder on modern CPUs than the lattice NTT inverse.

The asymmetry between Corona’s 5 M and Magnetar’s 30 000 – despite comparable CPU verify times – reflects two distinct calibrations:

- Corona’s verify dominates a single core because the NTT inverse + polynomial multiplication mod q over the larger Ring-LWE coefficients is a tight cache-resident loop with limited ILP. 1.6 ms on a single core is $\sim 141\times$ the Pulsar verify; the gas charge rounds to $\sim 100\times$.
- Magnetar’s verify is a hash-tree walk: hash function evaluation is highly parallelizable, well-vectorized by modern CPUs, and memory-bandwidth-bound. 1.92 ms wall-clock is dominated by sequential dependencies in the WOTS+ structure but the per-operation cost is lower than the lattice multiplication. The gas charge reflects the modest per-call CPU cost.

This is the same calibration discipline LP-218 uses for Pulsar: gas reflects CPU-class verifier cost, not GPU-class. Operators with all-Blackwell validator sets may petition for discounted gas in a future amendment; LP-220 does not codify it.

6.4 Validator registration

Each P3Q-conformant validator MUST register a `P3QVerifier` for every kind it will accept on-chain. Misconfiguration – a validator missing the Corona verifier when Z-Chain receives a rollup-batch tx that calls `P3Q.verify(0x02, ...)` – is a configuration error, not a soundness error: the validator’s EVM cannot validate the block, and consensus excludes that validator’s vote on the block. LP-202 tier-degradation handles the recovery (the validator votes `nil` on the block; consensus continues with the remaining validators).

The reference `p3q.Run` ships `KindPulsar` wired and `KindCorona` / `KindMagnetar` reserved-but-not-wired; this is the genesis state. As the Corona and Magnetar verifier libraries land in subsequent versions of `luxfi/corona` and `luxfi/magnetar`, their P3Q dispatch entries are wired in, and the wired-state advances. The wire format and the Solidity ABI are stable across the wired/not-wired transition; only the verify behavior changes (from `false` to the actual cryptographic outcome).

Failure	Trigger	Effect
Unknown kind byte	caller passes kind not in {0x01, 0x02, 0x03}	precompile returns abiFalse ; caller observes false
Reserved-but-not-wired kind	caller passes kind=0x02 or kind=0x03 before verifier lands	precompile returns abiFalse ; caller observes false
Malformed wire layout	caller passes truncated/oversized fields	precompile returns abiFalse
Sig-length mismatch (Pulsar)	sigLen $\neq$ FIPS-204 expected size for the mode byte	precompile returns abiFalse
FIPS verifier reject	sig genuinely does not verify under the group pubkey	precompile returns abiFalse
Out-of-gas	caller supplies gas < RequiredGas(input)	precompile returns contract.ErrOutOfGas ; caller transaction reverts
Group pubkey rotation skew	sequencer rotated keys without updating the on-chain group pubkey	precompile returns abiFalse until rotation tx lands

Table 11: Failure modes. Every cryptographic failure is **abiFalse**; only out-of-gas reverts. This is the LP-218 “no revert on cryptographic failure” contract – the Solidity caller decides whether **false** is fatal to its batch.

6.5 Failure modes

7 Composition With Sibling LPs

7.1 LP-217 cert profile modes and the P3Q kinds

The LP-217 cert-profile-mode ladder is the operator-facing naming for cross-family PQ posture. Each mode declares a required leg set; the P3Q kinds supply the on-chain machinery that makes each mode’s leg set callable from a rollup contract:

Mode	Required legs (P3Q calls)	Per-batch gas	Use case
PQ-off	BLS only (no P3Q)	~5000	non-financial; gaming
PQ-fast	BLS + P3Q(0x01)	~55 000	payments, DEX, gaming
PQ-strict	BLS + P3Q(0x01) + P3Q(0x02)	~5 055 000	custody, treasury, securities
PQ-heavy	BLS + P3Q(0x01) + P3Q(0x02) + P3Q(0x03)	~5 085 000	sovereign anchors, bridges

Table 12: LP-217 modes mapped to P3Q kind calls. The four cert profile modes correspond to subsets of the P3Q kind set; PQ-heavy calls all three. The ~5 M-gas cost cliff between PQ-fast and PQ-strict is the price of cross-family Ring-LWE insurance via the Corona kind.

The composition is **additive**: a PQ-heavy rollup contract calls the BLS precompile and three `P3Q.verify(kind, ...)` calls at slot 0x012205 in sequence inside its `validateBatch` function. If any of the four returns **false**, the batch is rejected.

7.1.1 Sample rollup-contract composition (Solidity)

```
function validateBatch(
    bytes32 prevRoot,
```



```

    bytes32 newRoot,
    bytes32 batchHash,
    bytes calldata blsSig,
    bytes calldata pulsarSig,
    bytes calldata coronaSig,    // omit for PQ-strict
    bytes calldata magnetarSig,  // omit for PQ-strict/fast
    bytes calldata groupPk
) external returns (bool) {
    bytes32 h = sha256(abi.encodePacked(prevRoot, newRoot, batchHash));

    // Classical leg
    if (!BLS.verify(blsSig, h, blsGroupPk)) return false;

    // P3Q Pulsar leg (PQ-fast and above)
    if (!P3Q.verify(0x01, pulsarSig, h, groupPk)) return false;

    if (mode >= PQ_STRICT) {
        // P3Q Corona leg (Ring-LWE family independence)
        if (!P3Q.verify(0x02, coronaSig, h, groupPk)) return false;
    }

    if (mode == PQ_HEAVY) {
        // P3Q Magnetar leg (hash-based family independence)
        if (!P3Q.verify(0x03, magnetarSig, h, groupPk)) return false;
    }

    return true;
}

```

The same precompile address (0x012205) is called three times with three different kind bytes. The rollup contract’s mode field is set at rollup-genesis time via `CertModeFloor` (LP-218); the contract enforces it on every batch.

7.2 LP-218 rollup-batch pattern

LP-218 specifies the rollup-batch envelope and the sequencer→Z-Chain submission pattern; LP-220 specifies the per-kind verifier mechanics. The two LPs share the same slot (0x012205), the same ABI shape (`P3Q.verify(kind, sig, h, pk)`), and the same `abiFalse`-on-cryptographic-failure contract.

LP-218 § “Precompile wire format” documents the structural format; LP-220 sections 3–5 document the per-kind specifics on top of that uniform skeleton. The intent is clean cross-reference rather than duplication: a reader who wants the rollup-batch pattern reads LP-218; a reader who wants per-kind verifier mechanics reads LP-220; a reader who wants both reads both.

7.3 LP-300 schema registry

The P3Q family lives at one slot, but the wire envelopes – the sequencer→Z-Chain `RollupBatchTx` payload, the inner-batch Pulsar/Corona/Magnetar signature blobs – are schema-versioned per LP-300. The registry entry for the P3Q family is:

```

slot:    0x012205
kinds:   { 0x01: pulsar, 0x02: corona, 0x03: magnetar }
domain:  lux-evm-precompile-p3q-v1    (FIPS 204 ctx)
abi:     P3Q.verify(uint8, bytes, bytes32, bytes) -> bool

```

Schema bumps within a kind (e.g. a future Corona parameter set revision) advance the mode byte without changing the slot or the kind byte. Schema bumps across the family (a fourth kind, say) advance the kind byte without changing the slot.

LP	Relationship
LP-073	Pulsar threshold signature – P3Q kind 0x01 verifies
LP-181	Magnetar threshold SLH-DSA – P3Q kind 0x03 verifies
LP-182	QuasarCert – the cert legs whose verify kinds LP-220 supplies
LP-200	ZAP Stack umbrella – the “one and only one way” principle the unified slot honors
LP-202	Cert tier degradation – the per-kind verify path participates in the tier ladder
LP-203	GPU-native verify – batched Corona / Magnetar kernels (projected)
LP-217	Cert profile modes – the operator-facing names that determine which P3Q kinds a rollup contract calls
LP-218	ZAP-native PQ rollups – specifies the unified slot and kind 0x01 Pulsar; LP-220 specifies kinds 0x02 Corona and 0x03 Magnetar on the same slot
LP-219	Rollup-to-rollup atomic – cross-rollup batches verify via the P3Q family at each participating rollup’s chosen kind set
LP-0120	Quasar mainnet defaults – owns slots 0x012204, 0x012206, 0x012207 for raw-primitive verifiers (the non-rollup-batch use case); LP-218/LP-220 P3Q family lives exclusively at 0x012205
LP-300	Schema registry – entry for the P3Q family kind set
FIPS 204	ML-DSA standard – Pulsar-kind verifier conformant
FIPS 205	SLH-DSA standard – Magnetar-kind verifier conformant
Ringtail eprint 2024/1113	Ring-LWE threshold reference – Corona-kind verifier conformant

Table 13: LP-220 composition references.

7.4 Cross-references

7.5 Backwards compatibility

None. The pre-Quasar Edition Lux network had no P3Q precompile family and is a separate network out of scope. The genesis state of the new final Lux network (activation 2025-12-25T16:20:00-08:00) ships the unified-slot kind-byte-dispatch ABI as the only ABI; no shim exists for the sibling-slots draft.

7.6 Future work

- **Corona/Magnetar verifier wiring.** The reference `p3q.Run` returns `abiFalse` for kinds 0x02 and 0x03 today. The follow-up is to wire `luxfi/corona/sign.Verify` and `luxfi/magnetar/verify` into the dispatch table. The ABI and the wire format are stable across this transition; only verify behavior changes from `false` to the cryptographic outcome.
- **GPU dispatch boundary closure.** Install `lux-lattice.pc` on Spark and expose `lux_slhdsa_*` / `lux_corona_*` entry points in `lux/accel/c_api.h`. Once both close, the Blackwell verify projections (sections 4 and 5) become measurements.
- **Fourth PQ family.** If a code-based signature scheme (HQC-Sig variants, McEliece-Sig) becomes operationally relevant, it lands as one new `P3QVerifier` impl and one new `kind` byte (provisionally 0x04) in the dispatch table. No new precompile slot allocation, no new Solidity function shape.
- **Blackwell-discounted Corona gas.** Operators with all-Blackwell validator sets may petition for a discounted Corona gas charge reflecting GPU-class verify cost rather than CPU-class; LP-220 does not codify this. A future amendment would tie the discount to a

validator-hardware attestation surface.

References

- [1] Bamboo, Cong Zhang, Tianhao Wang, et al. Ringtail: A two-round lattice threshold signature from ring-lwe, 2024. IACR ePrint 2024/1113. <https://eprint.iacr.org/2024/1113>.
- [2] Lux Industries. Lp-073: Pulsar two-round lattice threshold signing, 2026.
- [3] Lux Industries. Lp-181: Magnetar – public-dkg mpc threshold slh-dsa, 2026.
- [4] Lux Industries. Lp-200: The lux zap stack, 2026.
- [5] Lux Industries. Lp-203: Gpu-native verify, 2026.
- [6] Lux Industries. Quasar 4-Scheme Matrix Bench – m1 max + dgx spark gb10, 2026. `lux/threshold/protocols/parity/parity_test.go`, run 2026-06-04. Mean of $N=\{16, 32, 64\}$ across 50 rounds.
- [7] NIST. FIPS 205: Stateless hash-based digital signature standard (SLH-DSA), 2024. <https://csrc.nist.gov/pubs/fips/205/final>.